

BRECQ, a fundamental step in enabling post training quantization for low bit-widths

Daniel Dumschat¹

¹University of Applied Sciences Mittelhessen, Germany

Quantization is a widely used approach to reduce the memory footprint and inference cost of deep neural networks, enabling deployment on resource-constrained devices. Two primary strategies exist: quantization-aware training (QAT), which integrates quantization into the training loop but is compute- and time-intensive, and post-training quantization (PTQ), which adapts a pretrained model to reduced precision without full retraining. PTQ is attractive for its efficiency, but historically has been reliable mainly at relatively high bit-widths (e.g., 8 bits); aggressive low-bit quantization (4 bits, 2 bits) often led to large accuracy degradations or was infeasible for many architectures. BRECQ introduced a reconstruction-based methodology that substantially improves the robustness of PTQ at low bit-widths, thereby opening the door to practical, low-precision PTQ for a broader class of models. In this report, I (i) review the background and taxonomy of quantization methods, (ii) present a clear, technical exposition of the BRECQ approach and the principles that enable its success at low precisions, and (iii) describe a transfer study in which the core ideas from BRECQ are adapted to a newer generation of object detection architectures (YOLO family) that differ structurally from the networks emphasized in prior work. I conclude with empirical findings, limitations, and directions for extending low-bit PTQ to additional model families.

Date: June 26, 2026

Project page and code: <https://github.com/dumschat-daniel/Masterseminar-Code>

Correspondence: daniel.dumschat@mni.thm.de

Contents

1 Introduction	2
1.1 Fundamentals of Numerical Representation in Neural Networks	2
1.2 Fundamentals of Quantization	3
2 Methodology	4
2.1 Differences between QAT and PTQ	4
2.2 BRECQ	4
2.2.1 Previous Approaches	4
2.3 Block-wise Reconstruction	6
2.4 Mixed Precision	7
3 Results	8
4 Implementation	9
4.1 Fundamental Understanding	9
5 BRECQ Framework	14
6 Conclusion	16
6.1 Future Work	16
7 Acknowledgment	16
References	16
A Results	18

1 | Introduction

1.1 | Fundamentals of Numerical Representation in Neural Networks

Neural networks are fundamentally numerical systems that transform inputs into outputs through a sequence of arithmetic operations applied to large collections of numbers. Every parameter, intermediate activation, and output is represented in digital hardware using a finite number of bits. Consequently, the choice of numerical representation is not merely an implementation detail—it directly determines memory consumption, computational efficiency, and ultimately the practical deployability of a model. To understand why reduced-precision methods such as quantization are effective, it is necessary to first examine how numbers are represented in digital systems and how these representations interact with neural network computations.

All numerical values in modern computing systems are encoded in binary form. A representation using b bits can encode at most 2^b distinct values, which implies that only a finite subset of the real numbers can be represented exactly. This fundamental limitation forces any numerical format to approximate continuous values. The design of a numerical representation determines how these discrete bit patterns are assigned to values, governed by two central properties: the range of representable values and the precision with which they can be distinguished. Because the total number of bit patterns is fixed, any increase in range typically reduces

precision, creating a fundamental trade-off that underlies all practical number formats. [1]

The simplest mapping between bit patterns and numerical values is found in integer and fixed-point representations. Signed integers allow for the representation of whole numbers within a fixed range, while fixed-point formats extend this by introducing an implicit scaling factor, interpreted as $x = s \cdot q$. While these formats are highly efficient and well-supported in hardware, their rigid structure limits their dynamic range. This makes them less suitable for deep neural networks where values often vary by several orders of magnitude across different layers and training phases.

To address these limitations, floating-point representations introduce a dynamic scaling mechanism. A floating-point number is expressed as $x = (-1)^s \cdot m \cdot 2^e$, where the exponent e allows the decimal point to “float,” enabling a massive dynamic range that can represent both extremely small and very large magnitudes. This flexibility has made floating-point the standard choice for neural network training, as it provides the numerical stability required for gradients and weight updates to converge. However, this flexibility comes at a significant cost: floating-point arithmetic requires complex hardware and consumes substantially more memory than integer-based logic.

Neural networks have traditionally used 32-bit floating point (FP32) precision as the default numerical representation. FP32 can accurately represent both very small and very large numerical values, which is particularly important for gradient-based optimization where rounding errors may accumulate across millions or billions of operations. Consequently, most deep learning frameworks and hardware platforms historically adopted FP32 as the standard representation for model parameters, activations, and gradients [2]. This convention is closely tied to the IEEE 754 floating-point standard, which established single-precision floating point as a reliable and broadly supported numerical format for scientific computing and machine learning workloads.

Modern deep learning models require substantial compute and memory resources during both training and inference, since millions or billions of parameters and activations must continuously be transferred between memory hierarchies, caches, and compute units. Micikevicius et al. [3] note that overall deep learning performance is typically constrained by one or more of three factors: arithmetic throughput, memory bandwidth, or latency.

Reduced-precision computation directly addresses these limitations by decreasing both memory bandwidth pressure and arithmetic cost on hardware optimized for low-precision operations. Because fewer bits are required to represent each numerical value, reduced precision formats significantly lower memory footprint, reduce the amount of data transferred throughout the system, and improve computational throughput. A simple illustration can be found here: Listing 2

These principles form the foundation of neural network quantization, which reduces the numerical precision used to represent weights and activations in order to improve computational efficiency. Lower-precision representations reduce memory consumption and bandwidth requirements while enabling higher throughput on specialized accelerators such as GPUs and edge AI hardware [4]. Consequently, reduced precision arithmetic has become a critical optimization strategy for deploying modern deep learning systems efficiently. [5], [6]

1.2 | Fundamentals of Quantization

The previous section established two key observations: neural networks rely entirely on finite-precision numerical representations, and high-precision formats such as floating point introduce significant computational and memory costs. At the same time, these models exhibit a notable robustness to small numerical perturbations. Quantization builds directly on this tension between cost and tolerance by deliberately reducing numerical precision in a controlled manner.

At a high level, quantization replaces high-precision real-valued representations with a smaller, discrete set of values that can be encoded using fewer bits. Rather than allowing each parameter or activation to take on a continuum of possible values, quantization constrains them to a finite set of levels. The central question is therefore not whether information is lost—this is unavoidable—but how to structure this loss such that it minimally affects the model's behavior. A simple illustration can be found here Listing 1.

A useful way to view quantization is as a mapping between two numerical domains. The original domain consists of real-valued quantities, typically represented in floating point. The target domain consists of integers within a fixed range determined by the chosen bit-width. The role of the quantization scheme is to define how values are projected from the former into the latter, and how they are

approximately reconstructed when needed for computation.

In practice, most neural network quantization methods adopt a uniform, or linear, mapping. The real-valued range of interest is partitioned into evenly spaced intervals, each corresponding to a discrete integer value. Two parameters govern this transformation: a scaling factor, which determines the spacing between adjacent levels, and an offset, which aligns the discrete grid with the real-valued domain. Together, these parameters define how densely different regions of the value range are represented. Depending on how the offset is chosen, quantization can be symmetric—where the range is centered around zero—or asymmetric, where an additional zero-point allows the discrete range to better align with non-centered distributions. Asymmetric schemes are particularly useful for activations, which are often strictly non-negative, while symmetric schemes are commonly preferred for weights due to their simpler arithmetic. [7]

This seemingly simple mechanism introduces an important trade-off. If the representable range is chosen to be very wide, the spacing between quantization levels increases, reducing precision. Conversely, restricting the range improves precision within that range but risks discarding values that fall outside it. In real neural networks, where weight and activation distributions are often non-uniform and may contain outliers, selecting an appropriate range becomes a central challenge. Much of practical quantization can therefore be understood as carefully balancing resolution against coverage.

The consequences of this mapping manifest as two primary forms of approximation error. Before quantization, a fixed numerical range must be chosen that defines the smallest and largest values that can be represented after mapping to discrete levels. This range is often determined from the statistical distribution of the layer's weights or activations. However, neural network parameters are not always uniformly distributed: most values may cluster within a small interval, while a few rare “outlier” values can be much larger in magnitude. First, rounding a value to the nearest discrete level introduces a small perturbation, often modeled as noise. Second, values that lie outside the chosen range must be clipped, resulting in a potentially larger and more structured distortion. For example, if the quantization range is set to $[-1, 1]$, a value of 1.7 would be clipped to 1. While rounding errors tend to average out across large computations, clipping errors can disproportionately affect specific compo-

nents of the network, making range selection a critical design decision. In low-bit regimes (8-bits or lower), mitigating the influence of outliers becomes especially important.

A further distinction arises between quantizing weights and activations. Weights are fixed after training and can therefore be analyzed globally, whereas activations depend on input data and may vary significantly across samples. As a result, activation quantization typically relies more heavily on representative data and careful range estimation, making it a primary source of error in post-training settings.

Another important dimension of quantization is how these mapping parameters are shared. A single global mapping applied to an entire tensor is computationally efficient but may fail to capture variations in scale across different parts of the network. Allowing parameters to vary at a finer granularity—such as per channel—enables a more faithful representation of heterogeneous distributions, particularly in weight tensors where different filters may operate at different magnitudes. This illustrates a recurring theme: improved accuracy often comes at the cost of additional complexity. A simple example why capturing variations in layers matters is illustrated in Listing 4.

Once values are quantized, many neural network computations can be performed using low-precision integer arithmetic instead of full-precision floating-point operations. In practice, intermediate accumulations are often carried out in higher precision to avoid overflow, after which results are rescaled back into the target quantized format. Quantization therefore affects not only how values are represented, but also how computations are executed throughout the network [8].

2 | Methodology

2.1 | Differences between QAT and PTQ

A central distinction in the literature is whether quantization is incorporated during training or applied after a model has already been trained. These two paradigms—quantization-aware training (QAT) and post-training quantization (PTQ)—represent fundamentally different approaches to managing the trade-off between efficiency and accuracy.[9]

Quantization-aware training (QAT) treats quantization as part of the learning problem, allowing the model to internalize and compensate for approximation effects. In this paradigm, quantization effects are incorporated directly into the optimiza-

tion process. During training, the forward pass simulates discretization using “fake quantization,” where values are mapped to discrete levels but stored in floating point to maintain precision for weight updates. Because the rounding operation is non-differentiable, the backward pass typically relies on a straight-through estimator (STE) to approximate gradients. This allows the model to adapt its parameters in response to quantization-induced distortions. As a result, QAT typically achieves strong performance even at low bit-widths, as the learned representations become inherently robust to discretization effects.

However, this robustness comes at a cost. QAT requires access to the full training pipeline, including the original training data, substantial computational resources, and extended training time. For large-scale models, retraining or fine-tuning with quantization can be prohibitively expensive. Moreover, integrating quantization into the training process introduces additional complexity in both implementation and hyperparameter tuning, which can limit the practical applicability of QAT.

In contrast, post-training quantization (PTQ) treats quantization as a post hoc transformation, emphasizing efficiency and applicability over adaptability. PTQ applies quantization to an already trained floating-point model without modifying its parameters through retraining. Instead of adapting the model to the quantization process, PTQ adapts the quantization scheme to the model. This is typically achieved by determining suitable quantization parameters, such as scaling factors and ranges, through calibration on a small representative dataset. Calibration often employs techniques such as min-max estimation, percentile-based clipping, or entropy-based criteria to align the discrete grid with the model's value distributions. These calibration methods are computationally inexpensive and fast in practice because they only require a small number of forward passes on representative data, without modifying model parameters or performing backpropagation. The different methods are visualized in Listing 3

2.2 | BRECQ

2.2.1 | Previous Approaches

The appeal of PTQ lies in its ability to quantize a pretrained network without retraining the entire model, allowing a single floating-point network to be adapted quickly to multiple deployment scenarios. The central objective of PTQ is therefore to preserve the performance of the original floating-

point model as closely as possible after mapping its weights to a low-bit discrete set. [10]

A natural starting point is to minimize the error introduced directly in parameter space, through uniform quantization:

$$\min_{\hat{w}} \|\hat{w} - w\|_F^2 \quad \text{s.t.} \quad \hat{w} \in Q_{s,b}^{\text{sym}} \quad (1)$$

Here, w denotes the original floating-point weights, $\hat{w}$ the quantized weights, $\|\cdot\|_F$ the Frobenius norm¹, and $Q_{s,b}^{\text{sym}}$ that denotes the symmetric quantization grid with step size s and bit-width b . However, this objective only ensures that the quantized weights remain numerically close to the original weights; it does not guarantee that the resulting model remains close in terms of task performance. A simple example that trying to match the original weights as closely as possible does not guarantee the best result is illustrated in Listing 5. More faithfully, PTQ would like to solve

$$\min_{\hat{w}} \mathbb{E}[L(\hat{w})] \quad \text{s.t.} \quad \hat{w} \in Q_{s,b}^{\text{sym}} \quad (2)$$

but this objective is difficult to optimize directly without retraining.

Early work such as **DFQ (Nagel et al., 2019)** [11] attempted to preserve performance without using any calibration data. Instead, it relied on parameter-space manipulations such as *cross-layer equalization*, which balances weight ranges across adjacent layers, and *bias correction*, which compensates for systematic shifts induced by rounding. These methods were effective at 8-bit quantization, but they degraded severely at lower bit-widths. As later emphasized by BRECQ, the core reason is that a good approximation in parameter space is not necessarily a good approximation in model space: without data, one cannot ensure that the final task loss is preserved.

To address this limitation, **Nagel et al. (2020)** [12] introduced a data-driven PTQ method based on a small calibration set. Their key insight was to analyze quantization as a perturbation Δw of a pretrained model and approximate the loss increase using a second-order Taylor expansion:

$$\mathbb{E}[L(w + \Delta w)] - \mathbb{E}[L(w)] \quad (3)$$

$$\approx \Delta w^T g(w) + \frac{1}{2} \Delta w^T H(w) \Delta w \quad (4)$$

where Δw represents the perturbation introduced by quantization, $g(w) = \mathbb{E}[\nabla_w L]$ is the gradient

of the loss with respect to the weights, $H(w) = \mathbb{E}[\nabla_w^2 L]$ is the Hessian matrix of second derivatives, and $((\cdot)^T)$ denotes the transpose operation. Since a trained model is assumed to be close to a local minimum, the first-order term—the gradient—is taken to be negligible because the slope at a stationary point is zero/flat, leaving the second-order Hessian term as the dominant contribution to the change in loss.

$$\frac{1}{2} \Delta w^T H(w) \Delta w \quad (5)$$

Directly optimizing this quantity is infeasible because the full Hessian is too large to compute and store. Nagel et al. therefore introduced simplifying assumptions, most importantly that the Hessian can be treated as approximately layer-diagonal, meaning that second-order interactions between parameters in different layers are assumed to be small compared to interactions within the same layer. Under these assumptions, the second-order objective can be reduced to a layer-wise output reconstruction objective of the form

$$\min \mathbb{E}[\|\Delta z^l\|_F^2], \quad \Delta z^l = \hat{z}^l - z^l \quad (6)$$

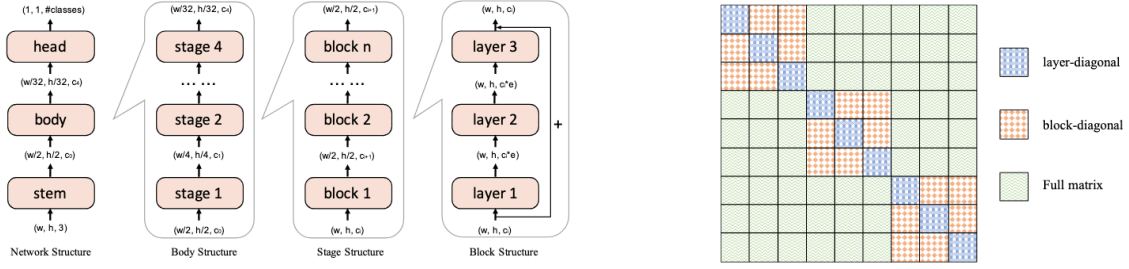
where z^l and $\hat{z}^l$ denote the floating-point and quantized outputs of layer l , respectively. Thus, each layer is quantized by minimizing the reconstruction error between the quantized output and its floating-point counterpart on calibration data.

To optimize the discrete rounding decisions, Nagel et al. further proposed **AdaRound**, which replaces fixed nearest rounding [13] with a learnable soft decision. Instead of always rounding to the nearest quantization point, each weight is parameterized as

$$\hat{w} = s \cdot \text{clip}\left(\left\lfloor \frac{w}{s} \right\rfloor + \sigma(v), n, p\right) \quad (7)$$

v is a learnable parameter that controls the rounding decision in a differentiable way. Instead of applying a fixed rule (always rounding up or down), the model adjusts v during training so that the final quantized weight best supports the reconstruction objective. $\sigma(v)$ is a smooth mapping (typically a sigmoid) that converts v into a value in the interval $(0, 1)$. This value acts as a soft indicator between rounding down and rounding up: values closer to 0 bias the result toward the lower quantization level, while values closer to 1 bias it toward the higher level. n and p define the lower and upper bounds of the valid quantization index range. After rounding is influenced by $\sigma(v)$, the result is clipped to ensure it stays within the representable discrete set. While

¹The Frobenius norm measures the overall magnitude of a matrix or tensor by summing the squares of all entries and taking the square root.



(a) A typical structure of CNN (taken from Radosavovic et al. (2020)) [17]. Network is composed of a stem layer (first convolution on input images), a body and a head layer (average pooling with a fully connected layer). A body contains several stages, and a stage contains several blocks. A representative block is the bottleneck block with residual path.

(b) An example illustration of Hessian (or Fisher) matrix. Blue sub-block means the layer-diagonal and each layer are mutual-independent; orange sub-block considers the dependency inside a building block and green parts measure all dependencies.

Figure 1: The 4 kinds of reconstruction granularity, namely net-wise, stage-wise, block-wise and layer-wise optimization, each of them corresponds an essential component of CNN. Taken directly from [10].

this layer-wise formulation marked a major step forward, it still ignored **cross-layer dependencies**. In practice, quantization noise introduced in one layer changes the input distribution of subsequent layers, so errors accumulate across the network. These dependencies become especially important at 4-bit and 2-bit precision, where the perturbation Δw is no longer small. As a result, methods that optimize each layer independently often suffer substantial accuracy loss at very low bit-widths.

At the other extreme, one could try to optimize the discrepancy at the level of the **entire network output**, analogous to a distillation-style objective [14], [15]. Rather than focusing on individual layers, this view measures quantization error by how much the **final prediction** of the network changes, thereby naturally capturing how errors propagate and interact across layers.

BRECQ formalizes this idea by showing that, under a Gauss-Newton approximation, the second-order error in parameter space can be rewritten in terms of the perturbation of the network output:

$$H(\theta) \approx J_{z^n}(\theta)^T H_{z^n} J_{z^n}(\theta) \quad (8)$$

where θ denotes the model parameters, $z^{(n)}$ is the intermediate network output (or representation) at layer/block n , $J_{z^n}(\theta)$ is the Jacobian of $z^{(n)}$ with respect to θ , and $H(\theta)$, H_{z^n} are the Hessians of the loss with respect to θ and $z^{(n)}$ respectively.

Moreover, since

$$\Delta z^n \approx J_{z^n}(\theta) \Delta \theta, \quad (9)$$

where $\Delta \theta$ and $\Delta z^{(n)}$ denote small perturbations in parameter space and representation space, it follows that

$$\Delta \theta^T H(\theta) \Delta \theta \approx \Delta z^{(n)T} H_{z^n} \Delta z^n \quad (10)$$

Intuitively, this means that the impact of quantization is determined not by how much individual weights change, but by how those changes affect the final output of the network. This provides a theoretical justification for output reconstruction as a proxy for preserving task performance.

However, directly optimizing the entire network output with only a small calibration set is impractical. In PQT, access to the original training data is often limited or unavailable, and calibration is therefore performed on a small unlabeled dataset. With many layers and limited data, jointly adjusting all quantization decisions leads to an underconstrained and unstable optimization problem that is prone to overfitting, since too many parameters are adjusted based on too little data, which enables the network to memorize, rather than generalize [16]. Layer-wise reconstruction acts like a regularizer to prevent that. Both approaches have their drawbacks. This motivates the central idea of BRECQ: instead of reconstructing each layer independently or the whole network jointly, quantization should be performed at an intermediate granularity. By operating on groups of layers, one can capture the most important cross-layer interactions while keeping the optimization problem tractable and stable.

2.3 | Block-wise Reconstruction

To address the limitations of both layer-wise and network-wise reconstruction, intermediate reconstruction granularities are introduced. In particular, block-wise reconstruction leverages the inherent hierarchical structure of convolutional neural networks (CNNs).

Modern CNN architectures are composed of nested components: layers are grouped into blocks (e.g., residual or bottleneck blocks), and blocks are further grouped into stages. A block typically consists of several convolutional layers combined with non-linearities and, in many cases, residual connections. These blocks form natural computational units in which layers exhibit strong mutual dependencies due to their sequential and often skip-connected structure. Figure 1a

Block-wise reconstruction treats each block as a single unit during quantization. Instead of reconstructing individual layers independently, all parameters within a block are optimized jointly. This allows intra-block dependencies to be preserved, while inter-block dependencies are still approximated as independent, as shown in Figure 1b. As a result, block-wise reconstruction provides a balance between the efficiency of layer-wise methods and the accuracy of network-wise optimization.

Formally, consider the output of a block at layer index l as $z^{(l)}$, and its quantized counterpart as $\hat{z}^{(l)}$. The reconstruction error is defined as:

$$\Delta z^{(l)} = \hat{z}^{(l)} - z^{(l)}. \quad (11)$$

A naïve approach would minimize the mean squared error:

$$\min_{\hat{w}} \mathbb{E} \left[\|\Delta z^{(l)}\|^2 \right], \quad (12)$$

which assumes that all output dimensions are equally important. However, this ignores the fact that different components of the block output can have vastly different impacts on the final task loss.

To account for this, a second-order approximation of the loss is considered:

$$\mathbb{E} \left[(\Delta z^{(l)})^\top H(z^{(l)}) \Delta z^{(l)} \right], \quad (13)$$

where $H(z^{(l)})$ denotes the Hessian of the loss with respect to the block output. This expression is the block-output analogue of the second-order Taylor approximation of the loss introduced earlier, where the perturbation is measured in activation space rather than directly in weight space [Equation 3]. This term captures how sensitive the final loss is to perturbations in each output direction.

Since computing the full Hessian is computationally infeasible, it is approximated using the diagonal of the Fisher Information Matrix (FIM). This leads to the objective:

$$\min_{\hat{w}} \mathbb{E} \left[\sum_i \left(\frac{\partial L}{\partial z_i^{(l)}} \right)^2 (\Delta z_i^{(l)})^2 \right]. \quad (14)$$

Where L is the final task loss of the entire network.

This formulation can be interpreted as a weighted reconstruction error, where each output dimension is scaled by the squared gradient of the loss with respect to that output. Intuitively, this assigns greater importance to components that have a larger influence on the final prediction.

From an optimization perspective, this approach addresses a key limitation of directly minimizing the final loss. While it would in principle be possible to evaluate the full model with quantized blocks and compare the resulting loss, such a strategy would not provide information about which specific components of the block output are responsible for performance degradation. Moreover, repeatedly propagating changes through the entire network would be computationally expensive.

By contrast, the gradient-based weighting provides a local proxy for global loss sensitivity. The gradients indicate how strongly each activation affects the final loss, effectively acting as a sensitivity map. This enables the reconstruction process to focus on preserving the most critical features of the block output, while allowing less important components to tolerate larger quantization errors.

In addition to block-wise reconstruction, stage-wise reconstruction is introduced as a coarser granularity. A stage consists of multiple consecutive blocks operating at the same spatial resolution. While block-wise methods capture dependencies within individual blocks, stage-wise reconstruction further accounts for interactions across blocks within the same stage, at the cost of increased computational complexity.

Overall, block-wise reconstruction represents a practical compromise: it respects the intrinsic structure of CNNs, captures meaningful intra-block dependencies, and incorporates task-aware sensitivity information, while remaining computationally efficient for post-training quantization and is therefore the chosen granularity by the authors.

2.4 | Mixed Precision

While the previously described reconstruction methods optimize the model within a fixed bit-width, mixed precision allows for varying bit-widths across different layers or blocks. This approach exploits the fact that neural network layers exhibit heterogeneous sensitivity to quantization;

some can tolerate drastic precision reduction with minimal impact on accuracy, while others are critical to the model’s predictive capability.

The objective is to find a bit-width configuration $c \in \{2, 4, 8\}^n$ that minimizes the task loss while adhering to a hardware budget δ :

$$\min_c L(\hat{w}, c) \quad \text{subject to} \quad H(c) \leq \delta. \quad (15)$$

Building on the Hessian-based sensitivity analysis used in block-wise reconstruction, the authors model the loss in two parts: a diagonal loss for independent layer sensitivity and an off-diagonal loss to account for cross-layer dependencies. Since a full combinatorial search across 3^n configurations is computationally prohibitive, the search space is constrained by focusing on the most critical transitions in precision. While independent layer optimization is already insufficient at 4-bit precision, the performance degradation becomes even more severe at 2-bit. The search strategy therefore prioritizes identifying which layers can tolerate 2-bit quantization and which must be kept at higher bit-widths (4-bit or 8-bit) to maintain stability.

To find the optimal configuration within this reduced space, a genetic algorithm is employed to navigate the hardware constraints. Intuitively, this turns quantization into a resource allocation problem: the available “bit budget” is concentrated on high-sensitivity blocks identified by the Fisher Information Matrix, while robust blocks are compressed further to meet the target efficiency. By integrating these variable bit-widths with second-order reconstruction, the framework achieves significant reductions in memory and latency while maintaining the robustness of the high-precision original model.

3 | Results

The proposed method is evaluated across standard CNN architectures, where block-wise reconstruction is used as the primary granularity, showing the highest accuracy compared to other granularities as shown in Table 1. This means that further grouping several blocks into stages hindered performance, as well as the previously discussed layer-wise optimization, which fails to capture necessary cross-layer dependencies, or whole-network optimization, which becomes susceptible to overfitting and optimization instability due to the high-dimensional parameter space. In practice, the first and last layers are treated separately and kept at higher precision, as they are known to be particularly sen-

sitive to quantization and have a disproportionate impact on overall accuracy.

Model	Layer	Block	Stage	Net
“ResNet-18”	65.19	66.39	66.01	54.15
“MobileNetV2”	52.13	59.67	54.23	40.76

Table 1: Ablation Study comparing different quantization granularities.

Across experiments, the Block method consistently outperforms existing post-training quantization approaches, particularly in low-bit regimes. For 4-bit weight quantization (4/32), the method achieves 70.70% on ResNet-18 and 76.29% on ResNet-50, closely matching full-precision performance (71.08% and 77.00%, respectively) and outperforming prior methods such as AdaRound (68.71%, 75.23%) and Bit-Split (69.11%, 75.58%).

The advantage becomes more pronounced at lower bit-widths. At 3-bit precision (3/32), the method maintains 69.81% accuracy on ResNet-18, while competing methods degrade more significantly. At 2-bit (2/32), where most post-training approaches fail (e.g., AdaRound drops to 55.96%), the proposed method still achieves 66.30%, demonstrating strong robustness in extreme quantization settings.

When both weights and activations are quantized, the method continues to outperform existing approaches. For 4/4 quantization, it achieves 69.60% on ResNet-18 compared to 67.5% (AdaQuant) and 67.56% (Bit-Split). Even at 2/4 precision, where most methods collapse to near-random performance, the method retains 64.80%, highlighting the effectiveness of the proposed reconstruction strategy.

Activations are primarily kept at 8-bit precision to ensure stability, as their distributions are input-dependent and more difficult to approximate. However, experiments show that reducing activations to 4-bit remains feasible with moderate accuracy loss, confirming that the method generalizes beyond conservative configurations.

Compared to quantization-aware training (QAT), the method achieves competitive performance without requiring full retraining. For example, on ResNet-18 with 4/4 precision, it achieves 69.60% accuracy using only 1024 calibration samples and 0.4 GPU hours, compared to LSQ which achieves 71.1% but requires 1.2M training samples and approximately 100 GPU hours. Overall, this translates to roughly two orders of magnitude less compute for producing a quantized model (about 240–250× in the reported setups), while staying close to QAT accuracy. Similar

trends are observed on MobileNetV2, where mixed-precision configurations reach up to 70.74% accuracy, closely matching QAT-based methods such as HAQ (70.90%). At the same time, mixed precision improves the accuracy–efficiency trade-off by allocating higher precision only where needed, achieving near-QAT performance at reduced model size.

Model	Parameters	Size (FP32)
ResNet-18	11,689,512	44.59 MB
ResNet-50	25,557,032	97.49 MB
MobileNetV2	3,504,872	13.37 MB

Table 2: Measured sizes of different Backbones.

In terms of deployment benefits, the method yields concrete reductions in weight storage relative to the FP32 baselines in Table 2. For instance, a 4/4 quantized ResNet-18 is reported at 5.81 MB (vs. 44.59 MB FP32) and MobileNetV2 at 2.26 MB (vs. 13.37 MB FP32), while a mixed-precision MobileNetV2 configuration further reduces weights to 1.38 MB. Inference efficiency is also evaluated via latency/throughput measurements under different precision assignments: notably, the reported latency is not strictly monotonic in bit-width because layer placement matters—e.g., an all-4-bit ResNet-18 configuration is reported at 53.28 ms, while a 2-bit configuration that keeps the first/last layers at higher precision is slower at 59.84 ms. This illustrates why mixed precision can outperform uniform low-bit quantization when optimizing under a fixed size or latency budget.

Finally, the method generalizes beyond classification tasks. On object detection benchmarks (MS COCO), it maintains near full-precision performance. For example, Faster R-CNN with a ResNet-18 backbone achieves 34.34 mAP at 4/8 precision compared to 34.55 in full precision, while significantly outperforming other PTQ methods under similar settings.

Overall, the results demonstrate that combining block-wise reconstruction with second-order sensitivity estimation enables accurate and robust post-training quantization across a wide range of models and bit-widths, while requiring only minimal data and computational resources. The entire results can be found in Appendix A.

4 | Implementation

THE FOLLOWING SECTION IS COMPLETELY NEW/CHANGED SO IT IS NOT COLORED.

The own implementation for this PTQ-focused literature research focused on exploring and understanding different parts of the quantization process and how and why information is stored and calculated the way it is. The repository containing all code snippets can be found here: <https://github.com/dumschat-daniel/Masterseminar-Code/tree/main>

All experiments are performed on my NVIDIA GeForce RTX 4060 Ti GPU.

4.1 | Fundamental Understanding

As discussed previously, quantization cannot preserve the complete information contained in the original model parameters. Instead, it is an information reduction technique that aims to retain as much relevant information as possible while representing it with fewer bits.

The following example demonstrates INT8 quantization applied to a small set of FP32 model weights:

```
# Example model weights
weights = np.array([0.12, -0.85, 1.43, -2.10, 0.57, 3.20],
                  dtype=np.float32)

# INT8 quantization
scale = np.max(np.abs(weights)) / 127
q_weights = np.round(weights / scale).astype(np.int8)

# Dequantization
dq_weights = q_weights.astype(np.float32) * scale

# Quantization error
mae = np.mean(np.abs(weights - dq_weights))
```

The quantization process produces the following results:

```
Original weights (FP32):
[0.12, -0.85, 1.43, -2.10, 0.57, 3.20]
Quantized weights (INT8):
[5, -34, 57, -83, 23, 127]
Recovered weights:
[0.126, -0.857, 1.436, -2.091, 0.580, 3.200]

Memory usage:
FP32 = 24 B
INT8 = 6 B

MAE: 0.00618
```

Quantization from FP32 to INT8 illustrates the trade-off between numerical precision and memory

efficiency. INT8 values are represented within the integer range of -127 to 127, while requiring only 8 bits per value compared to the 32 bits used by FP32 representations. This reduces the memory requirement by 75%.

After dequantization, the recovered weights closely approximate the original FP32 values. The resulting mean absolute error (MAE) of 0.00618 demonstrates that only a small amount of numerical precision is lost while achieving a significant reduction in memory usage.

However, this example only considers a small number of weights in a single layer. In large neural networks containing millions or billions of parameters across many layers, quantization errors can accumulate and propagate throughout the model. Consequently, the impact of information loss must always be evaluated in the broader context of overall model performance, accuracy requirements, and the computational efficiency gains achieved through quantization.

To further illustrate the impact of reduced numerical precision, a simple multilayer perceptron (MLP) was implemented and evaluated using both FP32 and FP16 representations. The experiment compares model memory usage, activation memory consumption, and inference latency to demonstrate the trade-offs between computational efficiency and numerical precision.

The following simplified implementation shows the core components of the experiment:

```
class SimpleMLP(nn.Module):

    def __init__(self):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(2048, 4096),
            nn.ReLU(),
            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.Linear(4096, 1024)
        )

    def forward(self, x):
        return self.net(x)

# FP32 model
model_fp32 = SimpleMLP().to(device).eval()
input_fp32 = torch.randn(64, 2048, device=device, dtype=torch.float32)

# FP16 model
model_fp16 = SimpleMLP().half().to(device).eval()
input_fp16 = input_fp32.half()

# Benchmark inference latency
fp32_latency = benchmark(model_fp32, input_fp32)
fp16_latency = benchmark(model_fp16, input_fp16)
```

The experiment produced the following representative results:

```
Model Parameter Memory:
FP32 Model Memory : 80.03 MB
FP16 Model Memory : 40.01 MB

Activation Memory:
FP32 Activations : 256.00 KB
FP16 Activations : 128.00 KB

Average Inference Latency:
FP32 Latency : 3.84 ms
FP16 Latency : 2.21 ms

Reduction Ratios:
Memory Reduction      : 2.00x
Activation Reduction   : 2.00x
FP16 Speedup          : 2.27x
```

The comparison between FP32 and FP16 demonstrates how reduced numerical precision can significantly decrease memory consumption while improving computational performance on modern GPUs. Since FP16 uses 16 bits instead of 32 bits per value, model parameters and activations require

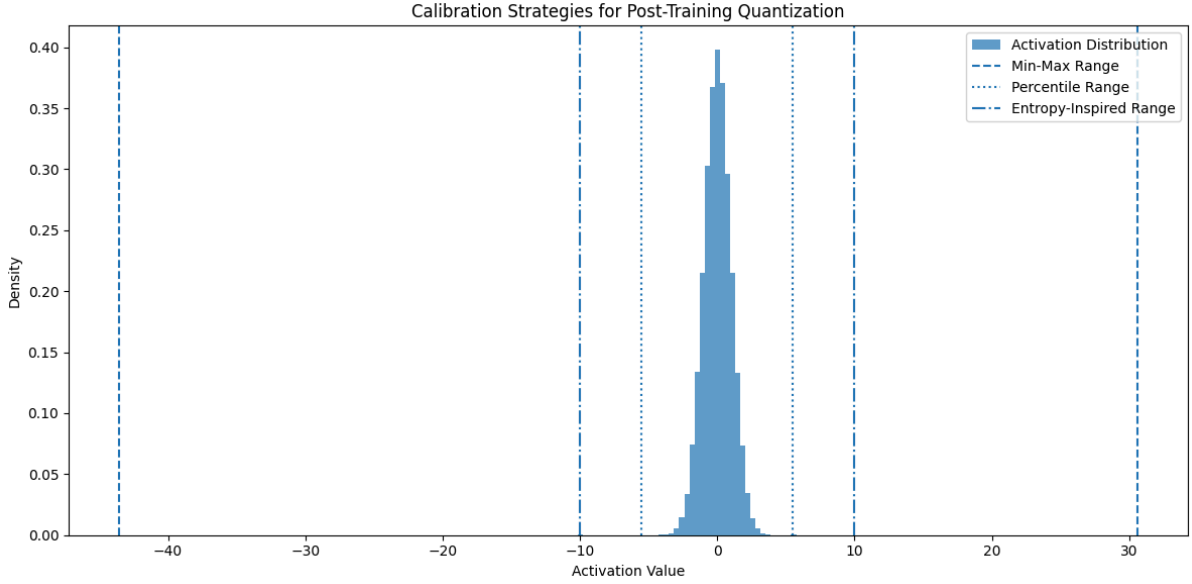


Figure 2: Visualization of different calibration ranges applied to a simulated neural network activation distribution for post-training quantization.

exactly half the memory compared to FP32 representations. However, certain computations and intermediate operations may still internally use FP32 arithmetic for numerical stability, meaning that the total runtime memory reduction in practical systems is not always exactly 50%.

In addition to reduced memory usage, FP16 computations can achieve lower inference latency due to improved hardware utilization on GPUs optimized for half-precision arithmetic. In this example, FP16 inference achieved a speedup of approximately $2.27\times$ compared to FP32 execution. This value varied from around $1.7\times$ to $2.5\times$ across different runs.

However, reduced precision also decreases the numerical accuracy available for representing model parameters and intermediate activations. While the performance benefits are substantial, rounding effects and reduced dynamic range can introduce numerical instability or accuracy degradation in larger and more complex neural networks. As model depth and parameter count increase, these effects may accumulate across layers and propagate throughout the network. Therefore, the impact of reduced precision must always be evaluated in the broader context of model accuracy, stability, and the computational efficiency gains achieved through lower-precision arithmetic.

To further analyze post-training quantization behavior, different calibration strategies were evaluated on a simulated neural network activation distribution. In practical neural networks, most activation values are concentrated near zero, while a small number of outliers can exhibit significantly

larger magnitudes. These outliers strongly influence the numerical range selected during quantization and therefore affect the resulting quantization error.

The following simplified implementation demonstrates three common calibration approaches used in post-training quantization:

```

# Simulated activation distribution
main_distribution =
np.random.normal(0.0, 1.0, 100000)
outliers = np.random.normal(0.0, 12.0,
300)

activations = np.concatenate([
    main_distribution,
    outliers
])

# Min-max calibration
minmax_min = activations.min()
minmax_max = activations.max()

# Percentile clipping
percentile_min =
np.percentile(activations, 0.1)
percentile_max =
np.percentile(activations, 99.9)

# Entropy-inspired calibration
candidate_thresholds = np.linspace(2.0,
10.0, 50)

for t in candidate_thresholds:

    result = quantize(activations, -t,
t)

    if result["mse"] < best_mse:
        best_threshold = t
        best_result = result

```

The calibration methods produced the following representative results:

```

1. Min-Max Calibration
Range: [-43.77, 30.83]
Scale: 0.2925
MSE: 0.0071

2. Percentile Clipping
Range: [-5.31, 5.28]
Scale: 0.0415
MSE: 0.1043

3. Entropy-Inspired Calibration
Range: [-9.80, 9.80]
Scale: 0.0769
MSE: 0.0297

```

Figure 2 illustrates the activation distribution together with the calibration ranges selected by the different methods.

Min-max calibration preserves the complete numerical range of the activations, including all extreme outliers. While this avoids clipping any values, it produces a very large quantization range and

therefore relatively coarse quantization steps. As a result, the precision for the majority of activation values near zero is reduced.

Percentile clipping instead ignores a small percentage of extreme outliers by selecting thresholds based on statistical percentiles. This significantly narrows the quantization range and increases precision for the majority of commonly occurring activation values. However, the clipped outliers introduce additional reconstruction error because values outside the selected range cannot be accurately represented.

The entropy-inspired calibration method attempts to find a balance between these two extremes. Rather than preserving all outliers or aggressively clipping them, it searches for a threshold that minimizes the overall quantization error. In practice, entropy- or KL-divergence-based calibration methods are widely used because they often provide better trade-offs between precision and clipping error for real neural network activations.

The figure demonstrates these differences visually. The min-max method selects the widest range in order to preserve all outliers, while percentile clipping chooses a much narrower interval focused on the dense central region of the distribution. The entropy-inspired method selects an intermediate range that balances clipping and numerical precision.

Although the calibration strategies differ in complexity and resulting quantization error, all post-training calibration approaches remain computationally far less expensive than retraining a model using QAT. However, the choice of calibration method can significantly influence the final accuracy of the quantized model, especially in deeper neural networks where quantization errors propagate across multiple layers.

Another important optimization technique in quantization is per-channel quantization. In many neural networks, different channels or layers can contain values with significantly different numerical ranges. Using a single global quantization range for all channels may therefore reduce precision for channels containing smaller values.

The following simplified example compares global quantization with per-channel quantization for two channels with very different magnitudes:


```

# Two channels with different ranges
weights = np.array([
    [0.01, 0.05, 0.08, 0.12], # small
    values
    [5.0, 8.0, 12.0, 15.0]    # large
    values
])

# Global quantization
global_min = weights.min()
global_max = weights.max()

global_scale = (global_max -
global_min) / 15

global_q = np.round(
    (weights - global_min) /
global_scale
)

global_dq = global_q * global_scale +
global_min

# Per-channel quantization
per_channel_dq = []

for channel in weights:

    cmin = channel.min()
    cmax = channel.max()

    scale = (cmax - cmin) / 15

    q = np.round((channel - cmin) /
scale)
    dq = q * scale + cmin

    per_channel_dq.append(dq)

```

The experiment demonstrates that global quantization struggles to accurately represent the small-valued channel because the quantization range is dominated by the much larger values of the second channel. For example, the original small-channel values [0.01, 0.05, 0.08, 0.12] were reconstructed through global quantization almost entirely as 0.01, showing that the quantization steps were too coarse to preserve smaller differences between the values.

In contrast, per-channel quantization reconstructed the same channel as [0.01, 0.0467, 0.0833, 0.12], which much more closely approximates the original distribution. This demonstrates how separate scaling factors allow smaller numerical ranges to retain significantly more precision while still using the same quantized bit width.

In practical neural networks, per-channel quantization is widely used for weights because it better

preserves information across layers and channels with different statistical properties. Although it introduces slightly more computational complexity and requires additional scaling parameters, it often yields significantly better accuracy compared to global quantization.

As discussed previously, minimizing the numerical difference between original and quantized weights does not necessarily produce the best task performance. Because neural networks are highly nonlinear systems, task loss and parameter reconstruction loss are not directly equivalent.

The following simplified experiment compares these two quantization strategies for a small neural network with interacting parameters:

```

# Method A:
# Select weights closest to FP32 values
best_param = min(grid,
    key=lambda w: parameter_distance(w))

# Method B:
# Select weights with lowest task loss
best_task = min(grid,
    key=lambda w: task_mse(w))

```

The trained FP32 model learned weights close to:

```

w1 = 3.01
w2 = 5.00

```

The resulting quantized solutions were:

```

Parameter-space quantization:
Weights: (3.5, 5.5)
Test MSE: 0.1314

Task-loss quantization:
Weights: (2.5, 5.5)
Test MSE: 0.0884

```

Although the parameter-space quantization selected weights numerically closer to the original FP32 values, it produced a higher task error. The task-loss optimized solution achieved approximately 1.49× lower test error despite being slightly further away in parameter space.

This demonstrates that preserving model behavior is often more important than preserving individual weight values exactly. In practical quantization methods, minimizing task loss is therefore typically more relevant than minimizing parameter reconstruction error alone.

5 | BRECQ Framework

The official BRECQ implementation provides a configurable framework for evaluating PTQ on pre-trained convolutional neural networks. The framework enables systematic experimentation with different quantization bit widths, reconstruction strategies, calibration methods, and precision configurations on ImageNet-based datasets. The implementation used in this work is publicly available through the official BRECQ repository at <https://github.com/yhhhli/BRECQ>.

The provided implementation is primarily optimized for ImageNet [18] classification tasks. Consequently, the framework assumes ImageNet-specific input dimensions of 224×224 pixels and internally uses layer groupings and reconstruction procedures designed for standard ImageNet architectures. For the experiments conducted in this work, the smaller Imagenette dataset [19] was used as a computationally lightweight subset of ImageNet.

Because the default ResNet-18 architecture provided by the framework is configured for the original 1000 ImageNet classes, the final classification layer had to be modified to support the 10 classes of Imagenette. Without fine-tuning, the pretrained ImageNet model achieved only a Top-1 accuracy of 6.96% and a Top-5 accuracy of 50.88% on Imagenette, which is expected due to the dataset mismatch.

To adapt the network to the new classification task, the model was fine-tuned on Imagenette for two epochs. After fine-tuning, the FP32 baseline model achieved a Top-1 accuracy of 97.65% and a Top-5 accuracy of 99.95%, providing a strong reference model for the subsequent quantization experiments.

Several quantization experiments were then conducted using different weight and activation bit widths, including 8-bit, 4-bit, and 2-bit configurations. The experiments aimed to evaluate the influence of reduced numerical precision on classification accuracy and to compare the robustness of different quantization settings.

Although the theoretical concepts of BRECQ are generally applicable to arbitrary architectures and datasets, the official implementation remains closely coupled to ImageNet-style workflows and supported network architectures. Furthermore, as noted by the original authors, achieving optimal quantization performance still requires careful tuning of reconstruction and calibration parameters for custom datasets and application domains.

To support Imagenette, minor modifications to the data loading and model initialization pipeline inside `main_imagenet.py` were necessary. In particular, the final fully connected classification layer was adapted from 1000 output classes to 10 output classes:

```
cnn = eval('hubconf.{}'.format(args.arch))
(pretrained=False)
cnn.fc = torch.nn.Linear(cnn.fc.in_features, 10)
cnn = cnn.cuda()
cnn.eval()
# -----
# load checkpoint
# -----
checkpoint_path = "resnet18_imagenette.pth"

checkpoint = torch.load(checkpoint_path,
                        map_location="cpu")

state_dict = checkpoint["model_state_dict"]

cnn.load_state_dict(state_dict,
                    strict=True)

cnn.eval()
```

The resulting fine-tuned FP32 model served as the baseline for all subsequent quantization experiments. It should be noted that Imagenette represents a substantially simpler classification problem than the full ImageNet dataset, and therefore relatively high quantized accuracies are expected even at lower bit widths.

Configuration	Top-1 Accuracy (%)	Top-5 Accuracy (%)
FP32 Baseline	97.648	99.949
W8A8	97.478	99.898
W4A4	97.274	99.898
W2A4	96.433	99.949
W4A2	89.146	99.490
W2A2	87.898	99.389

Table 3: The table shows the comparison of different bit-width configurations and their achieved Top-1- and Top-5 Accuracies.

Table 3 summarizes the primary quantization experiments performed using different weight and activation bit widths. All experiments were conducted using the default BRECQ reconstruction parameters and can be reproduced using the following command:

```
python3 main_imagenet.py \
  --arch resnet18 \
  --data_path /path/to/imagenette \
  --n_bits_w <n> \
  --n_bits_a <n> \
  --channel_wise \
  --act_quant
```

In this notation, W denotes the bit width used for weights, while A denotes the bit width used for activations.

The experimental results demonstrate that BRECQ maintains high classification accuracy even at substantially reduced precision levels. Both the W8A8 and W4A4 configurations achieved performance very close to the FP32 baseline, indicating that moderate quantization introduces only minimal accuracy degradation on the Imagenette dataset.

Compared to the FP32 baseline accuracy of 97.65%, the W8A8 configuration achieved a Top-1 accuracy of 97.48%, corresponding to a reduction of approximately 0.17 percentage points. Similarly, the W4A4 configuration achieved 97.27% Top-1 accuracy, representing a reduction of approximately 0.37 percentage points relative to the baseline. In both cases, Top-5 accuracy remained nearly unchanged, decreasing by only approximately 0.05 percentage points. These results indicate that both 8-bit and 4-bit quantization preserve almost the full predictive capability of the original FP32 network.

More noticeable degradation appears at extremely low bit widths. The W2A4 configuration reduced Top-1 accuracy from 97.65% to 96.43%, corresponding to a loss of approximately 1.22 percentage points. Despite this reduction, the Top-5 accuracy remained identical to the FP32 baseline at 99.95%, suggesting that the correct class still consistently remained among the highest-ranked predictions even when the top prediction became less reliable.

The largest degradation occurred for the W2A2 configuration, where both weights and activations were quantized to only 2 bits. In this case, Top-1 accuracy decreased by approximately 9.75 percentage points relative to the FP32 baseline, while Top-5 accuracy decreased by approximately 0.56 percentage points. Nevertheless, the model still retained compara-

tively strong classification performance despite the extremely limited numerical precision.

A particularly important observation is the influence of activation precision at ultra-low bit widths. Comparing W2A4 and W2A2 isolates the effect of reducing activation precision from 4 bits to 2 bits while maintaining a constant weight precision of 2 bits. This change alone reduced Top-1 accuracy by approximately 8.54 percentage points and Top-5 accuracy by approximately 0.56 percentage points. These results strongly support the conclusions presented in the original BRECQ paper, which emphasized the importance of maintaining higher activation precision when operating at aggressively quantized weight precisions. The other way around using W4A2 shows only slightly higher Top-1 accuracy compared to the W2A4 counterpart.

W8A8 Variant	Top-1 Accuracy (%)	Top-5 Accuracy (%)
Standard W8A8	97.478	99.898
W8A8 without Channel-wise	97.452	99.898
W8A8 with 128 Calibration Samples	97.452	99.898

Additional ablation experiments shown in Table 4 investigated the influence of channel-wise quantization and calibration dataset size on W8A8 performance. The first ablation disabled channel-wise quantization, while the second reduced the number of calibration samples from the default value of 1024 to only 128 samples.

Both modifications resulted in only minimal performance degradation. Disabling channel-wise quantization reduced Top-1 accuracy by approximately 0.03 percentage points, while reducing the calibration dataset size produced an identical reduction. Top-5 accuracy remained unchanged across all evaluated configurations.

These results demonstrate the robustness of the BRECQ reconstruction framework under moderate configuration changes. At the same time, they also suggest that the strong FP32 baseline and the comparatively low complexity of the Imagenette dataset limited the observable sensitivity to calibration and reconstruction variations. More challenging datasets and architectures would likely expose

larger differences between these quantization configurations.

6 | Conclusion

BRECQ represents a fundamental contribution to the field of PTQ. The paper demonstrates for the first time that PTQ can achieve performance comparable to QAT at 4-bit precision while remaining effective even at 2-bit quantization levels, where earlier PTQ approaches typically failed. The central contribution of the framework is the introduction of block-wise reconstruction, which minimizes quantization error locally across network blocks and thereby stabilizes ultra-low-bit quantization without requiring expensive end-to-end retraining.

A major advantage of BRECQ is its ability to retain the practical benefits of PTQ while significantly improving low-bit accuracy. Because the framework avoids full retraining and instead performs localized reconstruction optimization, the authors report model generation speedups of up to approximately 240× compared to conventional QAT approaches. This substantially lowers the computational and engineering requirements for deploying compressed neural networks on resource-constrained hardware.

The paper further demonstrates that carefully optimized reconstruction and calibration strategies are essential for maintaining stability at extremely low precisions. In particular, the results highlight the importance of activation precision and layer-wise reconstruction in preserving model performance under aggressive quantization settings.

The experiments conducted in this work using the Imagenette-Dataset support the findings presented in the original paper. In particular, the evaluated BRECQ configurations maintained high classification accuracy at 8-bit and 4-bit precision with only minimal degradation relative to the FP32 baseline, while lower-bit configurations still retained comparatively strong performance. The experiments additionally confirmed the strong influence of activation precision on model accuracy at ultra-low bit widths.

6.1 | Future Work

Although BRECQ remains highly influential in computer vision quantization research, more recent approaches have further extended its reconstruction and optimization concepts, particularly for ultra-low-bit quantization regimes. Methods such as QDrop [20] improve reconstruction robustness through stochastic quantization strategies and

demonstrate how BRECQ’s core principles continue to influence current PTQ research.

At the same time, modern quantization research is increasingly shifting toward Transformer architectures and Large Language Models (LLMs) [21], where efficient inference has become a major practical challenge due to rapidly growing model sizes. This has resulted in the development of specialized PTQ pipelines and calibration strategies tailored specifically to Transformer-based architectures.

Future research will likely continue focusing on improving quantization robustness under extremely low precision, reducing calibration data requirements, and developing data-free or synthetic-data calibration methods. Additional research directions include mixed-precision quantization, hardware-aware quantization optimization, and integration with other compression techniques such as pruning [22], knowledge distillation [14], low-rank factorization [23], and compact neural network architecture design [24].

More broadly, the optimal deployment strategy for efficient neural networks will likely continue to depend heavily on the target application and deployment constraints. PTQ remains especially attractive in scenarios requiring fast deployment, low engineering overhead, and limited retraining resources, whereas QAT continues to provide advantages when maximum achievable accuracy justifies the additional training cost and computational complexity.

7 | Acknowledgment

This work’s writing is primarily based on, and illustrating works off, the paper: BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction and their python implementation: <https://github.com/yhhhli/BRECQ>.

References

- [1] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.
- [2] D. Khudia *et al.*, “FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference.” [Online]. Available: <https://arxiv.org/abs/2101.05615>
- [3] P. Micikevicius *et al.*, “Mixed Precision Training.” [Online]. Available: <https://arxiv.org/abs/1710.03740>
- [4] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A White Paper on Neural Network Quantization.” [Online]. Available: <https://arxiv.org/abs/2106.08295>
- [5] D. D. Lin, S. S. Talathi, and V. S. Annapureddy, “Fixed Point Quantization of Deep Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1511.06393>

- [6] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep Learning with Limited Numerical Precision," in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., in *Proceedings of Machine Learning Research*, vol. 37. Lille, France: PMLR, Jul. 2015, pp. 1737–1746. [Online]. Available: <https://proceedings.mlr.press/v37/gupta15.html>
- [7] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep Learning with Limited Numerical Precision." [Online]. Available: <https://arxiv.org/abs/1502.02551>
- [8] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [9] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference." [Online]. Available: <https://arxiv.org/abs/2103.13630>
- [10] Y. Li *et al.*, "BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction." [Online]. Available: <https://arxiv.org/abs/2102.05426>
- [11] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, "Data-Free Quantization Through Weight Equalization and Bias Correction." [Online]. Available: <https://arxiv.org/abs/1906.04721>
- [12] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, "Up or Down? Adaptive Rounding for Post-Training Quantization." [Online]. Available: <https://arxiv.org/abs/2004.10568>
- [13] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "PACT: Parameterized Clipping Activation for Quantized Neural Networks." [Online]. Available: <https://arxiv.org/abs/1805.06085>
- [14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network." [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [15] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization." [Online]. Available: <https://arxiv.org/abs/1802.05668>
- [16] D. Jakubovitz, R. Giryes, and M. Rodrigues, "Generalization Error in Deep Learning." p. , Aug. 2018. doi: 10.48550/arXiv.1808.01174.
- [17] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing Network Design Spaces." [Online]. Available: <https://arxiv.org/abs/2003.13678>
- [18] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [19] J. Howard, "Imagenette: A Smaller Subset of 10 Easily Classified Classes from ImageNet." [Online]. Available: <https://github.com/fastai/imagenette>
- [20] X. Wei, R. Gong, Y. Li, X. Liu, and F. Yu, "QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization." [Online]. Available: <https://arxiv.org/abs/2203.05740>
- [21] J. Hasan, "Optimizing Large Language Models through Quantization: A Comparative Analysis of PTQ and QAT Techniques." p. , Nov. 2024. doi: 10.48550/arXiv.2411.06084.
- [22] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both Weights and Connections for Efficient Neural Networks." [Online]. Available: <https://arxiv.org/abs/1506.02626>
- [23] R. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting Linear Structure Within Convolutional Networks for Efficient Evaluation." [Online]. Available: <https://arxiv.org/abs/1404.0736>
- [24] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." [Online]. Available: <https://arxiv.org/abs/1704.04861>

A Results

The results are taken directly from the original paper.

Methods	Bits (W/A)	ResNet-18	ResNet-50	MobileNetV2	RegNet-600	RegNet-3.2GF	Mnas- Net-2.0
Full Prec.	32/32	71.08	77.00	72.49	73.71	78.36	76.68
Bias Correction*	4/32	50.43	64.64	62.82	67.09	71.73	72.31
OMSE (Choukroun et al., 2019)	4/32	67.12	74.67	—	—	—	—
AdaRound (Nagel et al., 2020)	4/32	68.71	75.23	69.78	71.97*	77.12*	74.87*
AdaQuant (Hubara et al., 2020)	4/32	68.82	75.22	44.78	—	—	—
Bit-Split (Wang et al., 2020)	4/32	69.11	75.58	—	—	—	—
BRECQ	4/32	70.70$\pm$0.07	76.29$\pm$0.04	71.66$\pm$0.04	73.02$\pm$0.09	78.04$\pm$0.04	76.00$\pm$0.02
Bias Correction*	3/32	12.85	7.97	10.89	28.82	17.95	40.72
AdaRound (Nagel et al., 2020)*	3/32	68.07	73.42	64.33	67.71	72.31	69.33
AdaQuant (Hubara et al., 2020)*	3/32	58.12	67.61	12.56	—	—	—
Bit-Split (Wang et al., 2020)	3/32	66.75	73.24	—	—	—	—
BRECQ	3/32	69.81$\pm$0.05	75.61$\pm$0.09	69.50$\pm$0.12	71.48$\pm$0.07	77.22$\pm$0.04	74.58$\pm$0.08
Bias Correction*	2/32	0.13	0.12	0.14	0.18	0.11	0.11
AdaRound (Nagel et al., 2020)*	2/32	55.96	47.95	32.54	25.66	24.70	30.60
AdaQuant (Hubara et al., 2020)*	2/32	0.30	0.49	0.11	—	—	—
BRECQ	2/32	66.30$\pm$0.12	72.40$\pm$0.12	59.67$\pm$0.13	65.83$\pm$0.13	73.88$\pm$0.14	67.13$\pm$0.13

Table 5: Accuracy comparison on *weight-only quantized* post-training models. Activations here are unquantized and kept full precision. We also conduct variance study for our experiments. Bold values indicates best results. * indicates our implementation based on open-source codes.

Methods	Bits (W/A)	ResNet-18	ResNet-50	MobileNetV2	ResNet-600	RegNet-3.2GF	MNAS- Net-2.0
Full Prec.	32/32	71.08	77.00	72.49	73.71	78.36	76.68
ACIQ-Mix (Banner et al., 2019)	4/4	67.0	73.8	—	—	—	—
ZeroQ (Cai et al., 2020)*	4/4	21.71	2.94	26.24	28.54	12.24	3.89
LAPQ (Nahshan et al., 2019)	4/4	60.3	70.0	49.7	57.71*	55.89*	65.32*
AdaQuant (Hubara et al., 2020)	4/4	67.5	73.7	34.95*	—	—	—
Bit-Split (Wang et al., 2020)	4/4	67.56	73.71	—	—	—	—
BRECQ	4/4	69.60$\pm$0.04	75.05$\pm$0.09	66.57$\pm$0.67	68.33$\pm$0.28	74.21$\pm$0.19	73.56$\pm$0.24
ZeroQ (Cai et al., 2020)*	2/4	0.08	0.08	0.10	0.10	0.05	0.12
LAPQ (Nahshan et al., 2019)*	2/4	0.18	0.14	0.13	0.17	0.12	0.18
AdaQuant (Hubara et al., 2020)*	2/4	0.21	0.12	0.10	—	—	—
BRECQ	2/4	64.80$\pm$0.08	70.29$\pm$0.23	53.34$\pm$0.15	59.31$\pm$0.49	67.15$\pm$0.11	63.01$\pm$0.35

Table 6: Accuracy comparison on *fully quantized* post-training models. Activations here are quantized to 4-bit. Notations follows the upper table.

Models	Methods	Prec.	Accuracy	Model Size	Train Data	GPU hrs
ResNet-18 (FP: 71.08)	ZeroQ (Cai et al., 2020)	4/4	21.20	5.81 MB	0	0.008
	BRECQ	4/4	69.60	5.81 MB	1024	0.4
	BRECQ (w/ distilled data)	4/4	69.32	5.81 MB	0	0.4
	PACT (Choi et al., 2018)	4/4	69.2	5.81 MB	1.2 M	100
	DSQ (Gong et al., 2019)	4/4	69.56	5.81 MB	1.2 M	100
	LSQ (Esser et al., 2020)	4/4	71.1	5.81 MB	1.2 M	100
MobileNetV2 (FP: 72.49)	BRECQ	4/4	66.57	2.26 MB	1024	0.8
	PACT (Choi et al., 2018)	4/4	61.40	2.26 MB	1.2 M	192
	DSQ (Gong et al., 2019)	4/4	64.80	2.26 MB	1.2 M	192
	BRECQ	Mixed/8	70.74	1.38 MB	1024	3.2
	HAQ (Wang et al., 2019)	Mixed/8	70.90	1.38 MB	1.2 M	384

Table 7: Performance as well as training cost comparison with quantization-aware training (QAT).

Models	Backbone	Full Prec. 32/32	Bias Corr.* 8/8 4/8		AdaRound* 8/8 4/8		Ze- roQ 2/8 4 _{MP} /8		BRECQ 8/8 4/8	
2/8										
Faster RCNN (Ren et al., 2015)	ResNet-18	34.55	34.30	0.84	33.96	23.01	—	34.53	34.34	31.82
	ResNet-50	38.55	38.25	0.25	37.58	19.63	—	38.54	38.29	34.23
	MobileNetV2	33.44	33.24	18.39	32.77	16.35	—	33.40	33.18	27.54
RetinaNet (Lin et al., 2017)	ResNet-18	33.20	33.00	0.04	32.59	19.93	—	33.14	33.01	31.42
	ResNet-50	36.82	36.68	0.07	36.00	19.97	33.7	36.73	36.65	34.75
	MobileNetV2	32.63	32.60	18.47	31.89	14.10	—	32.57	32.31	27.59

Table 8: Object detection task (MS COCO) comparison on fully quantized post-training models. Activations here are quantized to 8-bit. We report the bounding box mean Average Precision (mAP) metric.